Deep Learning Bootcamp

CO3133 – Deep Learning and Applications

Weight: 40% of the course grade | Maximum: 40 points

Due: Sunday, 13 December 2026

Lecture

The Lecture going in-depth for this Assignment will be hosted next Wednesday - 28/10/2026.

Context and objective

Deep learning research involves more than training an existing model and reporting its score. It requires identifying a meaningful problem, proposing a justified method, designing fair experiments, and communicating what the evidence supports. This project takes you through that complete process, from finding data to writing a research-style paper.

Your task: Choose your own research topic in **computer vision or natural language processing (including LLMs)**. A vision–language topic is also welcome, but combining both modalities is not required. Define a clear input, output, use case, and research question.

Your contribution must be closely related to a deep learning model. Examples include modifying an architecture, designing a model ensemble or aggregation mechanism, changing a learning objective, developing a task-specific adaptation strategy, or compressing/distilling a model. Simply wrapping an existing model in an application is insufficient.

Demonstrate a clear strength through evidence. Your method may improve overall predictive performance, handle a specified difficult case more reliably, generalize better, or reduce parameters, memory, or inference time while retaining useful performance. State the strength precisely, compare against a suitable baseline, and report trade-offs. A larger model or a higher score on one selected example is not sufficient evidence.

1 Step 1: Pick a dataset and formulate the problem

Choose a dataset that supports a meaningful task and is large and diverse enough for your proposed training and evaluation. Adequacy depends on task difficulty, label quality, class balance, and whether pretrained models are used. Justify your choice. Avoid using too simple datasets like MNIST or CIFAR.

Look up in here to get a few good datasets to start with (Fact: you can search up Awesome Lists for pretty much anything programming).

- Report the source, usage conditions, collection process, sample count, labels, class distribution, and representative examples. Inspect the actual data.
- Identify duplicates, missing or noisy labels, imbalance, and potential biases. Check feasibility against your computing budget.
- Read relevant papers and identify a concrete limitation to investigate. Clearly distinguish your proposed contribution from existing work.
- State a testable hypothesis: for example, “Our modification improves minority-class F1 over baseline X under the same training budget.”

Checkpoint: A clear problem statement, dataset justification, baseline, and measurable claim.

2 Step 2: Design preprocessing and training

Specify your preprocessing, data split, model, loss function, optimizer, and training procedure. Explain *why* each choice fits the problem; listing settings alone is insufficient.

- Describe normalization, resizing or tokenization, augmentation, and handling of missing values or class imbalance, as applicable.
- Separate training, validation, and test data. Where samples are related, split by person, document, recording, product, location, or time. Keep different versions of the same sample in one split.
- Fit data-dependent preprocessing on training data only. Use validation data to choose hyperparameters and checkpoints; reserve the test set for final evaluation.
- Specify pretrained weights, frozen/trainable components, loss, optimizer, learning-rate schedule, batch size, epochs, stopping rule, and random seeds.

Checkpoint: A reproducible training protocol with an explicit explanation of leakage prevention.

3 Step 3: Select metrics and explain what they capture

Choose metrics that directly test your claimed strength. Explain their meaning, whether higher or lower is better, and which important errors they may overlook.

- **Prediction tasks:** Use appropriate metrics such as precision, recall, macro-F1, detection mAP, or segmentation IoU. Accuracy alone may hide poor minority-class performance.
- **Generative tasks:** Several wordings may be correct. Use the dataset's established evaluation protocol where available, and distinguish answer correctness from text similarity. For open-ended outputs, supplement automatic metrics with a defined human rubric for factuality, relevance, and coverage on a stated sample.
- **Efficiency claims:** Report parameters, latency, memory, or throughput alongside predictive quality. Specify hardware, input size, batch size, and measurement procedure.
- **Fair comparison:** Evaluate all methods on the same test data using the same metric implementation and settings. A similarity score is not a percentage of factual correctness.

Checkpoint: A primary metric, supporting metrics, and an explanation of their limitations.

4 Step 4: Set up experiments for generalization

Cover as many *relevant* conditions as feasible. Prioritize experiments that challenge your claim over a large number of arbitrary model runs.

- Compare against a credible existing method and the unmodified base model under comparable data and training conditions. Explain unequal resource budgets.
- Conduct **ablation studies**: remove or change proposed components to identify which ones cause the observed benefit.
- Test relevant classes, difficult cases, noise levels, data sizes, or unseen groups/domains. Define subsets before examining final results; justify any unavailable evaluations.
- Where feasible, repeat key experiments with multiple seeds and report variability. Show representative failures as well as successes.

Checkpoint: An experiment matrix linking each comparison to a research question. Keep claims within the scope of the evidence.

5 Step 5: Implement and run your code

Learn to use **Jupyter Notebook, NumPy, pandas, PyTorch or TensorFlow, and Matplotlib**, as appropriate. You do not need to use every library; select the tools your project needs and understand your implementation.

- Build a working baseline first, then implement your contribution. Verify data loading, tensor shapes, losses, and evaluation before expensive training.
- Keep data preparation, training, and evaluation clearly organized. Record configurations, software versions, seeds, hardware, and results.
- Provide a README with data-access instructions, environment setup, and commands or notebook execution order for reproducing the main results.
- Your codebase must contain your own implementation work. Reusing properly licensed frameworks and credited baseline code is allowed; clearly mark reused and modified components.

Checkpoint: A runnable codebase and traceable results, not only screenshots of notebook outputs.

6 Step 6: Write the research report

Learn to use **LaTeX and Overleaf**. Submit an **English report in the Springer LNCS format**. Use the official template without changing its margins or font sizes. The purpose of writing in English is to strengthen your technical communication skills for the workforce.

If writing in English is difficult, you may draft in Vietnamese and use AI to translate or improve the language. You remain responsible for technical accuracy, references, and every claim. Check the translated text carefully and disclose AI assistance.

Organize the report around: **Abstract; Introduction and contributions; Related work; Proposed method; Experimental setup; Results and analysis; Limitations; Conclusion; References**. Include an architecture diagram or equations where useful, clear comparison tables, and an honest account of unsuccessful experiments. Explain why your method works, where it fails, and what the results do not establish.

LNCS resources: Springer author guidelines and templates; LNCS template on Overleaf.

Submission requirements

Submit by **Sunday, 13 December 2026**:

1. **English report:** A PDF in LNCS format describing the problem, contribution, protocol, results, and limitations.
2. **Original codebase:** Your implementation, reproducible training/evaluation instructions, model checkpoints, dependency information, and configurations. Provide data-access and checkpoint instructions where needed, while respecting dataset licenses.

No-gos and academic integrity

- **Proprietary models as the experimental solution:** Do not feed project data directly into ChatGPT or another closed model/API to obtain task predictions or replace the proposed method. Use models whose weights and implementation you can access and evaluate reproducibly, subject to their licenses. AI translation of the report is the separate permitted use described above.
- **Copying an existing idea or project entirely:** A reproduction may be a baseline, but the submitted project needs a clearly identified contribution of your own. Changing only the dataset or interface is not, by itself, a model-related contribution.
- **Misleading evidence:** No fabricated results, uncited copying, test-set tuning, or selective reporting that hides relevant failures or unfair comparisons.

Assessment and rubric

Assessment follows the principles used in research-paper review: significance and originality of the contribution, technical soundness, strength of evidence, clarity, and reproducibility. No single benchmark score determines the grade.

30/40 and above represents exceptionally high-quality work that could be suitable for submission to a top tier research conference. It requires a convincing contribution, rigorous experiments, and polished presentation. This is a quality benchmark, not a promise of conference acceptance or a requirement to submit externally.

Category	Points	Assessment criteria
Method	**16**	**Problem and motivation (4):** Meaningful task, clear research question, and justified dataset. **Contribution (6):** Distinct, well-motivated model-related idea, positioned against prior work. **Technical soundness (6):** Correct design and implementation; justified architecture, training choices, and trade-offs.
Experiment	**16**	**Protocol and baselines (5):** Fair comparisons, suitable metrics, and leakage-free evaluation. **Evidence and generalization (7):** Demonstrated strength, informative ablations, relevant test conditions, and variability where feasible. **Analysis (4):** Interpretation of results, error analysis, limitations, and reproducible experimental records.
Presentation	**8**	**Report quality (5):** Clear English, coherent argument, informative figures/tables, correct citations, and LNCS formatting. **Code presentation (3):** Organized code, clear README, environment and execution instructions, and attribution of reused resources.
Total	**40**	**Contributes 40% of the final course grade.**

What earns credit: A carefully justified, well-tested improvement is stronger than an ambitious claim with weak evidence. A smaller model may be a strong contribution if its efficiency advantage and quality trade-offs are demonstrated fairly. Negative findings should be reported honestly; they earn credit for sound analysis, but do not establish an unproven claimed strength.

Topics from past students

- Integrating Unimodal and Vision-Language Representations in Latent Space for Multi-Label Chest X-Ray Classification
- A Hybrid Filtering Approach for Enhanced 3D Motion Tracking (ICIC 2026)
- Inference Gap Mitigation in Deep Fiducial Marker Pipelines through Jitter - Resilient Regression and Attention-Guided Decoding: Inference Gap Mitigation in Deep Fiducial Marker Pipelines (CSONet 2026)
- Photometric-consistent 4D Gaussian Splatting for Multimodal Sensor Alignment in Dynamic Multi-Person Scenes (CSONet 2026)

Still no idea? Talk to me! Bring a domain you find interesting, a candidate dataset, or a problem you would like to solve so we can discuss a feasible direction.

Sample papers

- InstructVideo: Instructing Video Diffusion Models with Human Feedback
- UniDepth: Universal Monocular Metric Depth Estimation
- CAD-SIGNet: CAD Language Inference from Point Clouds using Layer-wise Sketch Instance Guided Attention
- The Impact of Auxiliary Patient Data on Automated Chest X-Ray Report
- Generation and How to Incorporate It

Look up more on papers in common AI conferences like CVPR, ICCV, ECCV, ACL, EMNLP. ICLR, ICML, NeurIPS, etc.

Read the sample papers to study how authors formulate a problem, motivate a contribution, design comparisons, and connect conclusions to evidence. Use them as examples of research practice, not as projects to copy.

Support and Opportunities

This project is designed as a three-month 'bootcamp': an opportunity to develop an idea, test it, and improve it through experimentation and feedback. To support promising work, the following opportunities are available:

- **Access to domain specialists.** If your project requires expertise beyond the course material, I might be able to help connecting you with relevant specialists where available (some examples: Medical AI, applied maths, concurrency & sequential programming, etc.). Bring a clear problem statement and specific questions so that these discussions can be productive.
- **Computing support for promising teams.** During the project, I will select the top one or two teams showing strong potential and personally fund additional computing resources. Selection will consider the quality and feasibility of the idea, demonstrated progress, preliminary evidence, and a clear explanation of how additional computing resources would advance the work.

These opportunities are intended to help you develop your ideas further. Computing support is not a prerequisite for a high grade: all projects will be assessed using the same rubric, with attention to sound reasoning, fair experiments, and the evidence supporting your claims.